DA5402: Graduate Machine Learning Operations

Assignment 7: The 48-Hour MLflow Sprint

MNIST Digit Classifier with MLflow Lifecycle Management

Muhammed Fiyas

Roll no : DA25M018

April 14, 2026

Abstract

This report documents the implementation of a complete MLOps pipeline for MNIST handwritten digit classification using MLflow. The project covers the full model lifecycle: experiment tracking with hyperparameter logging, packaging the training script as an MLflow Project, serving the best model via MLflow's built-in REST API, and building a Streamlit web client for interactive inference. The best model achieved a validation accuracy of **99.05%** using a CNN architecture trained with a learning rate of 0.001 and batch size of 128.

Contents

1 Introduction **3**

2 Project Structure **3**

3 Dataset: MNIST **3**

4 Model Architecture **4**

5 Experiment Tracking **5**
5.1 MLflow Configuration 5
5.2 MLflow Logging Code 5
5.3 Experiment Results 5
5.4 Detailed Run Comparison 6

6 MLflow Projects **7**
6.1 MLproject File 7
6.2 Running the Project 7
6.3 How MLflow Maps runs:/ URI to the Filesystem 8

7 Model Serving **8**
7.1 Serving the Best Model 8
7.2 REST API Request Format 8
7.3 Sample API Response 8

8 Web Client **9**
8.1 Architecture 9
8.2 Running the Web Client 9

9 Conclusion **9**

Appendix: AI Disclosure **11**

1 Introduction

Machine Learning Operations (MLOps) bridges the gap between model development and production deployment. This assignment implements a 48-hour sprint to build a complete MLflow-based pipeline for MNIST digit classification. The key objectives are:

- Track experiments using MLflow's Python API
- Package the training script as a reproducible MLflow Project
- Serve the best model via MLflow's built-in REST API
- Build a Streamlit web client that communicates with the serving endpoint

2 Project Structure

```
assignment-7-muhammed786fiyas/
          MLproject                # MLflow project entry point
          conda.yaml               # Environment definition
          requirements.txt         # Pip dependencies
          app/
                app.py               # Streamlit web client
          src/
                train.py             # Training + MLflow tracking
                predict.py           # Inference helper
                preprocess.py        # Image preprocessing
                logger.py            # Structured logging
          notebooks/
              exploration.ipynb  # EDA notebook
```

3 Dataset: MNIST

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels.

Split	**Samples**	**Purpose**
Training	48,000	Model learning
Validation	12,000	Hyperparameter tuning
Test	10,000	Final evaluation

Table 1: MNIST Dataset Split

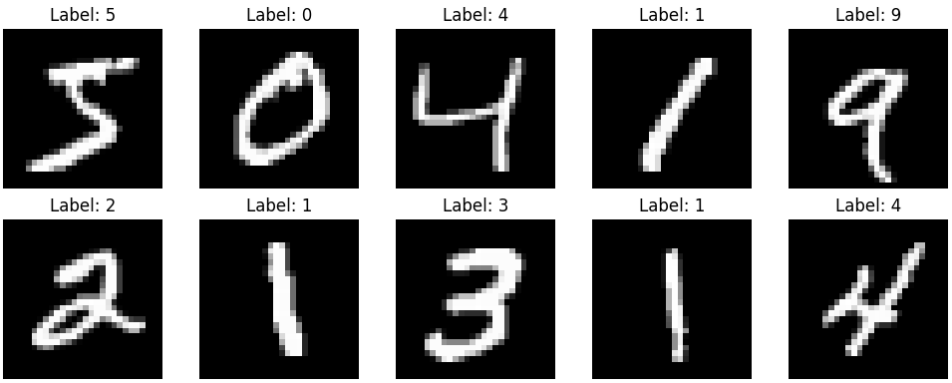


Figure 1: Sample images from the MNIST dataset

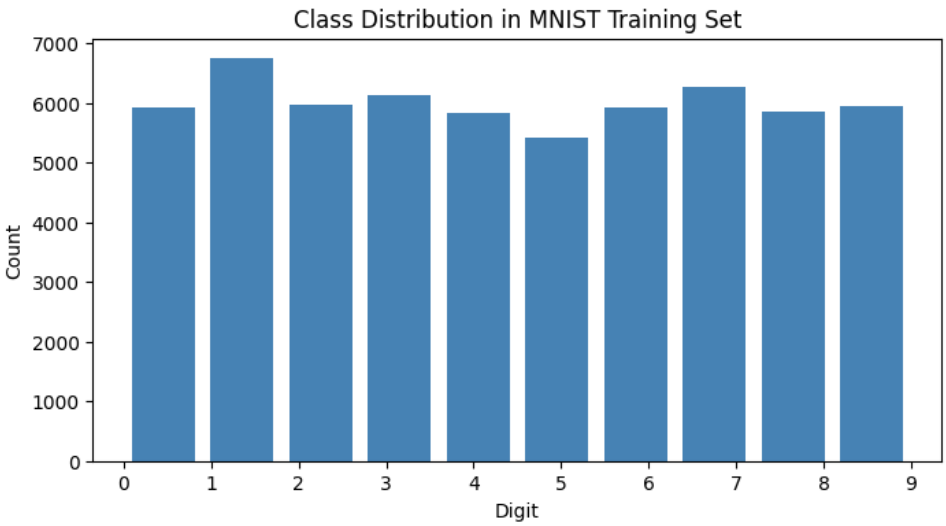


Figure 2: Class distribution in MNIST training set

4 Model Architecture

A Convolutional Neural Network (CNN) was designed for digit classification:

Layer	Type	Output Shape	Details
1	Conv2d + ReLU	(32, 28, 28)	32 filters, 3×3 kernel
2	MaxPool2d	(32, 14, 14)	2×2 pool
3	Conv2d + ReLU	(64, 14, 14)	64 filters, 3×3 kernel
4	MaxPool2d	(64, 7, 7)	2×2 pool
5	Flatten	(3136,)	
6	Linear + ReLU	(128,)	
7	Dropout	(128,)	p=0.3
8	Linear	(10,)	Output logits

Table 2: CNN Architecture (CNN-2Conv-2FC)

The model uses:

- **Loss function:** Cross Entropy Loss
- **Optimizer:** Adam
- **Dropout:** 0.3 to prevent overfitting

5 Experiment Tracking

5.1 MLflow Configuration

All experiments are tracked under the `MNIST-Classifier` experiment. Each run logs:

- **Parameters:** `learning_rate`, `batch_size`, `epochs`, `model_architecture`, `optimizer`
- **Metrics (per epoch):** `train_loss`, `train_accuracy`, `val_loss`, `val_accuracy`
- **Artifacts:** Trained model, prediction plot

5.2 MLflow Logging Code

```
with mlflow.start_run(run_name=f"lr{lr}_bs{bs}_ep{epochs}") as run:
    # Log parameters
    mlflow.log_param("learning_rate", learning_rate)
    mlflow.log_param("batch_size", batch_size)
    mlflow.log_param("model_architecture", "CNN-2Conv-2FC")

    # Log metrics per epoch
    mlflow.log_metric("val_accuracy", val_acc, step=epoch)
    mlflow.log_metric("val_loss", val_loss, step=epoch)

    # Log model artifact
    mlflow.pytorch.log_model(model, artifact_path="model")
```

5.3 Experiment Results

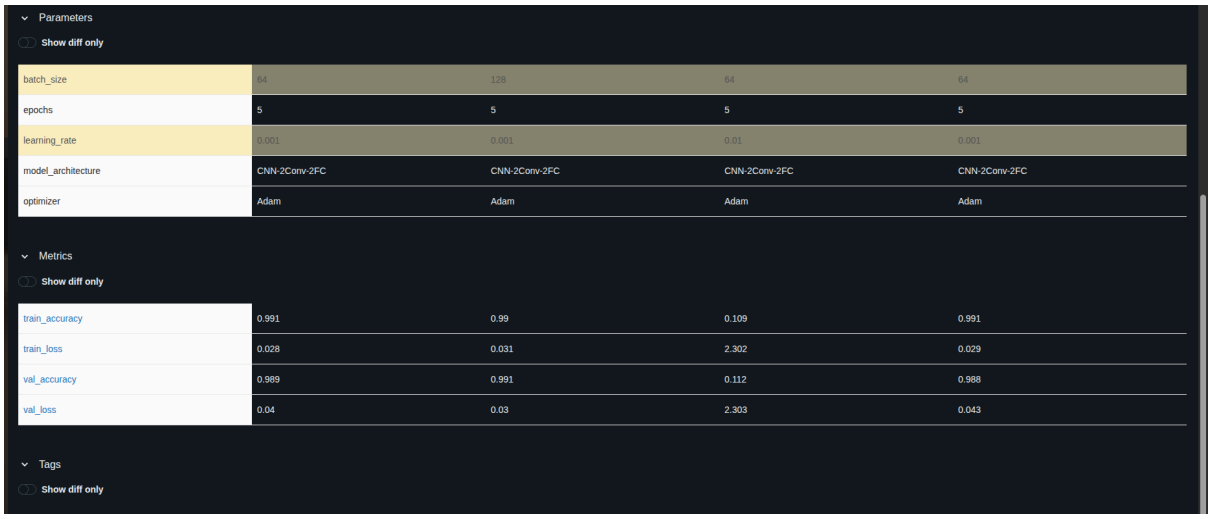
Three hyperparameter configurations were evaluated:

Run Name	LR	Batch	Val Accuracy	Val Loss
lr0.001_bs64_ep5	0.001	64	98.92%	0.0396
lr0.001_bs128_ep5	0.001	128	**99.05%**	**0.0301**
lr0.01_bs64_ep5	0.01	64	11.16%	2.3028

Table 3: Hyperparameter Experiment Results

The run with `lr=0.01` failed to converge, demonstrating that a high learning rate causes training instability for this architecture. The best run was `lr0.001_bs128_ep5` with **99.05%** validation accuracy.

5.4 Detailed Run Comparison



Parameters

Show diff only

batch_size	64	128	64	64
epochs	5	5	5	5
learning_rate	0.001	0.001	0.01	0.001
model_architecture	CNN-2Conv-2FC	CNN-2Conv-2FC	CNN-2Conv-2FC	CNN-2Conv-2FC
optimizer	Adam	Adam	Adam	Adam

Metrics

Show diff only

train_accuracy	0.991	0.99	0.109	0.991
train_loss	0.028	0.031	2.302	0.029
val_accuracy	0.989	0.991	0.112	0.988
val_loss	0.04	0.03	2.303	0.043

Tags

Show diff only

Figure 3: MLflow UI — Detailed parameter and metric comparison across all 4 runs

Key observations:

- **batch_size** and **learning_rate** are highlighted in yellow — MLflow automatically highlights parameters that differ across runs
- Run with `lr=0.01` clearly failed — `val_accuracy=0.112` and `val_loss=2.303`
- Best run (`lr=0.001, bs=128`) achieved `val_accuracy=0.991` and `val_loss=0.03`

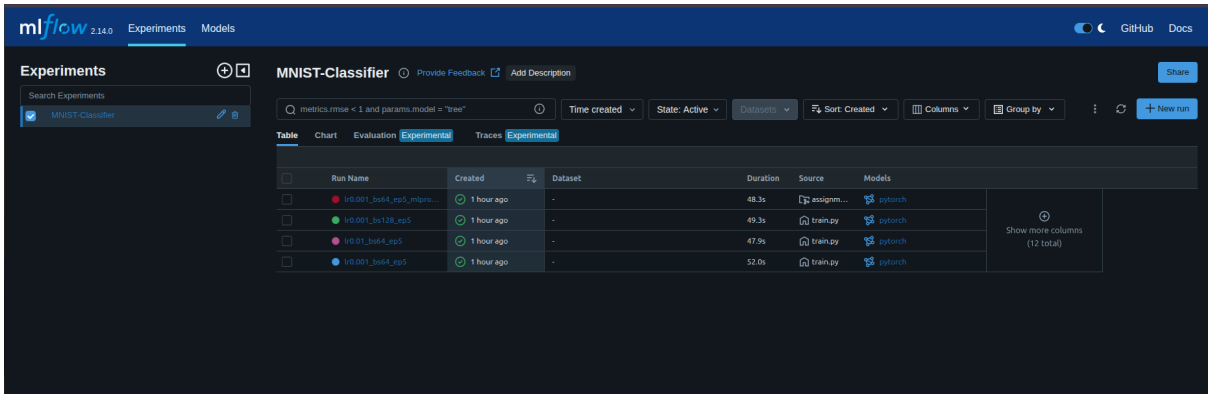


Figure 4: MLflow UI — Experiment comparison table

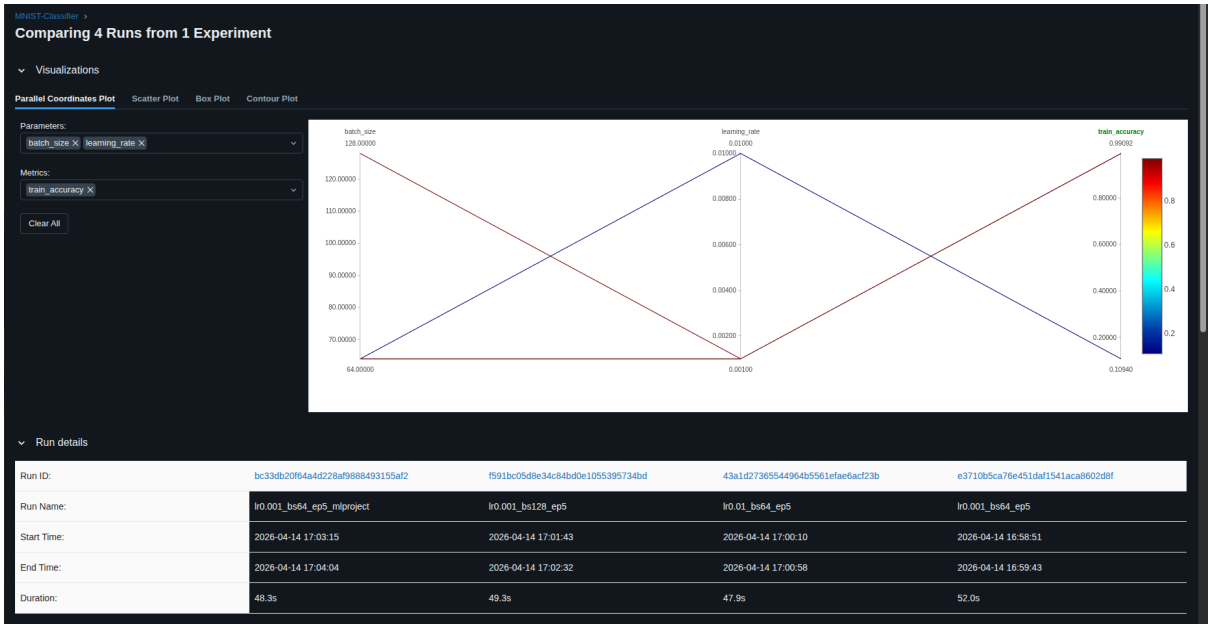


Figure 5: MLflow UI — Parallel coordinates comparison plot

6 MLflow Projects

6.1 MLproject File

```
name: mnist-classifier
conda_env: conda.yaml

entry_points:
  main:
    parameters:
      learning_rate: {type: float, default: 0.001}
      batch_size:    {type: int,   default: 64}
      epochs:        {type: int,   default: 5}
    command: "python src/train.py
              --learning_rate {learning_rate}
              --batch_size {batch_size}
              --epochs {epochs}"
```

6.2 Running the Project

```
mlflow run . -P learning_rate=0.001 -P batch_size=64 -P epochs=5 \
  --experiment-name MNIST-Classifier \
  --env-manager=local
```

6.3 How MLflow Maps runs:/ URI to the Filesystem

When MLflow logs a model, it stores it under:

```
mlruns/<experiment_id>/<run_id>/artifacts/model/
```

The `runs:/<run_id>/model` URI is resolved by MLflow's artifact registry which looks up the `run_id` in the local `mlruns/` directory, finds the corresponding `meta.yaml`, and resolves the artifact path to the local filesystem. This allows the same URI to work both locally and on remote tracking servers.

7 Model Serving

7.1 Serving the Best Model

```
mlflow models serve \
  -m "runs:/f591bc05d8e34c84bd0e1055395734bd/model" \
  --port 5001 \
  --env-manager=local \
  --no-conda
```

7.2 REST API Request Format

```
payload = {"inputs": tensor.float().numpy().tolist()}
response = requests.post(
    "http://127.0.0.1:5001/invocations",
    headers={"Content-Type": "application/json"},
    data=json.dumps(payload)
)
# Response: {"predictions": [[logit_0, logit_1, ..., logit_9]]}
```

7.3 Sample API Response

For the digit **7**:

```
{
  "predictions": [[
    -7.66, -3.33, -2.11,  0.37, -2.94,
    -5.56, -15.25, 15.09, -6.44,  0.35
  ]]
}
```

The highest logit (15.09) corresponds to class 7, confirming a correct prediction.

8 Web Client

8.1 Architecture

The Streamlit web client follows this flow:

1. User uploads a digit image via the file uploader
2. Image is preprocessed: grayscale → resize to 28×28 → normalize → tensor
3. Tensor is converted to a JSON payload and sent to MLflow REST API
4. API returns raw logits which are converted to probabilities via softmax
5. Predicted digit, confidence score, and probability distribution are displayed

8.2 Running the Web Client

```
streamlit run app/app.py
```

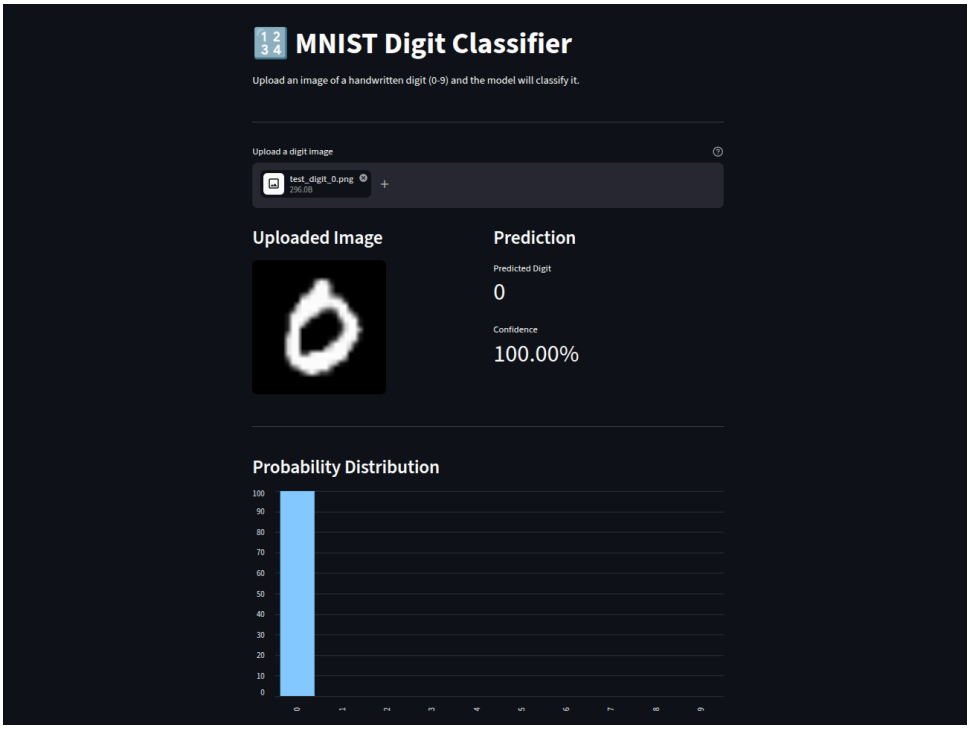


Figure 6: Streamlit web client showing digit classification result

9 Conclusion

This assignment successfully implemented a complete MLOps pipeline using MLflow:

- **Experiment Tracking:** 4 runs logged with parameters, metrics, and artifacts
- **MLflow Projects:** Reproducible training via `mlflow run .`
- **Model Serving:** REST API serving predictions on port 5001

- **Web Client:** Streamlit app with real-time digit classification

The best model achieved **99.05% validation accuracy** with `lr=0.001` and `batch_size=128`. The experiment also demonstrated that a high learning rate (`lr=0.01`) causes training instability, reducing accuracy to only 11.16%.

Appendix: AI Disclosure

Tool Used:

- Claude (Anthropic) — used as a coding and debugging assistant throughout the assignment

Nature of Prompts Provided:

- Environment setup and dependency resolution (conda, pip, torch CPU installation)
- Debugging assistance for Python errors (e.g., NumPy version incompatibility with torchvision, `pkg_resources` module errors, MLflow version conflicts)
- Clarification of MLflow concepts including experiment tracking API, `runs:/` URI resolution, and `MLproject` file structure
- Assistance in structuring the preprocessing pipeline and inference logic
- Guidance on MLflow REST API payload format and float32/float64 type mismatch debugging
- Requests to generate boilerplate code for the Streamlit web client

Sections Where AI Output Was Incorporated:

- **src/preprocess.py** — AI assisted in structuring the image preprocessing pipeline and `tensor_to_list()` conversion function
- **app/app.py** — Initial boilerplate for the Streamlit web client was AI-generated, then reviewed and customised (permitted under the assignment's Zero-Logic Clause)
- **src/logger.py** — AI assisted in designing the per-script logging utility with append mode and session separators
- **Debugging** — AI assisted in resolving environment dependency conflicts across MLflow, PyTorch, and NumPy versions
- **Report** — AI assisted in structuring and improving the clarity of this written report